

T.C.
BİLECİK ŞEYH EDEBALI ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



**Öneri Sistemlerinde Derecelendirme Tahmini: Varyasyonel Otokodlayıcı ile Matris
Faktörizasyonunun Karşılaştırmalı Performans Analizi Raporu**

Muhammed Ali Ezgin
50674850364

DANIŞMAN
Dr. Öğr. Üyesi Alper Yargıç

BM400 Bitirme Çalışması Rapor

BİLECİK 2026

BİLDİRİM

Bu çalışmada bütün bilgilerin etik davranış ve akademik kurallar çerçevesinde elde edildiğini ve yazım kurallarına uygun olarak hazırlanan bu çalışmada bana ait olmayan her türlü ifade ve bilginin kaynağına eksiksiz atıf yapıldığını bildiririm.

DECLARATION

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all materials and results that are not original to this work.

İmza

Muhammed Ali Ezgin

Tarih: 30 Ocak 2026

T.C.
BİLECİK ŞEYH EDEBALI ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

**Öneri Sistemlerinde Derecelendirme Tahmini: Varyasyonel Otokodlayıcı ile Matris
Faktörizasyonunun Karşılaştırmalı Performans Analizi Raporu**

Muhammed Ali Ezgin

Jüri Üyeleri

İmza

Danışman: Dr. Öğr. Üyesi Alper Yargıç

Jüri Üyesi 2: Prof. Dr. Adı SOYADI

Jüri Üyesi 3: Prof. Dr. Adı SOYADI

ÖZET

Öneri Sistemlerinde Derecelendirme Tahmini: Varyasyonel Otokodlayıcı ile Matris Faktörizasyonunun Karşılaştırmalı Performans Analizi

Bu çalışma, çeşitli platformlardan alınan gerçek sayısal puanlama bilgileri doğrultusunda, derin öğrenme tabanlı model olan varyasyonel otokodlayıcı ile geleneksel yöntem olan matris faktörizasyonunun tahmin performanslarını kıyaslamayı amaçlıyor. Veri seti olarak MovieLens veri seti seçilmiş olup seyrek biçimli kullanıcı-öge etkileşim matrisi üzerinden icra edilmiş, kaliteyi arttırmak için en az etkileşim koşulunu karşılamayan veriler çıkartılarak veri seyrekliği optimize edilmiştir. Matris faktörizasyonu tekniği için tekil değer ayrışımı modeliyle varyasyonel otokodlayıcı'nın yeniden parametrelendirme ve KL kaybı birleşimi olan kendine özgü yapısı denenmiştir. Bu kullanılan algoritmaların başarısı, kök ortalama kare hata, ortalama mutlak hata ve öneri listelerinin uygunluğunu ölçen normalize edilmiş indirgenmiş kümülatif kazanç metrikleriyle analiz edilmiştir. Elde edilen bulgularla iki yaklaşımın puan tahminindeki göreceli başarısını deneysel olarak analiz etmiştir.

Anahtar Kelimeler: Derin Öğrenme, Öneri Sistemleri, Varyasyonel Otokodlayıcılar, Matris Faktörizasyonu, Derecelendirme Tahmini

ABSTRACT

Comparative Performance Analysis of Variational Autoencoder and Matrix Factorization in Rating Prediction for Recommender Systems

This study aims to compare the prediction performances of the deep learning-based model, variational autoencoder, and the traditional method, matrix factorization, based on real numerical rating data obtained from various platforms. The MovieLens dataset was selected as the dataset and the study was executed over a sparse user-item interaction matrix, to enhance quality, data sparsity was optimized by removing data that did not meet the minimum interaction requirement. For the matrix factorization technique, the singular value decomposition model was tested against the unique structure of variational autoencoder, which is a combination of reparameterization and KL loss. The success of these algorithms was analyzed using root mean square error, mean absolute error, and normalized discounted cumulative gain metrics, which measure the suitability of recommendation lists. The findings experimentally analyze the relative success of the two approaches in rating prediction.

Keywords: Deep Learning, Recommender Systems, Variational Autoencoder, Matrix Factorization, Rating Prediction

ÖNSÖZ

Bitirme çalışmamın başından sonuna kadar emeği geçen ve beni bu konuya yönlendiren saygı değer hocam ve danışmanım Sayın Dr. Öğr. Üyesi Alper Yargıç'a tüm katkılarından ve hiç eksiltmediği desteğinden dolayı teşekkür ederim.

İmza

Muhammed Ali Ezgin

Tarih: 30 Ocak 2026

İÇİNDEKİLER

ÖZET	iv
ÖNSÖZ	vi
İÇİNDEKİLER	vii
ŞEKİLLER DİZİNİ	ix
TABLolar DİZİNİ	x
1 GİRİŞ	1
1.1 Literatür Taraması	1
2 KULLANILAN YAZILIMLAR ve YÖNTEMLER	4
2.1 Kullanılan Kütüphaneler	4
2.2 Veri Ön İşleme	5
2.3 Geleneksel Yöntem	5
2.4 Derin Öğrenme Yöntemi	5
2.5 Model Performans Metrikleri ve Hesaplamalar	6
3 VAE ve MF Tabanlı Karşılaştırmalı Öneri Sistemi	9
3.1 Veri Ön İşleme ve Matris Oluşturma	9
3.2 Matris Faktörizasyonu Modeli	10
3.2.1 MF Modeli Eğitimi ve Performans Metrikleri	11
3.2.2 MF için NDCG@K Veri Hazırlama Aşaması	12
3.3 Varyasyonel Otokodlayıcı Modeli	12
3.3.1 Öğrenme Oranı (Learning Rate) Analizi	12
3.3.2 Mimari Kapasite ve Gizli Boyut Analizi	13
3.3.3 Beta Parametresi ve Düzenleme Etkisi	13
3.3.4 Dropout Oranı ile Model Genelleştirme	14
3.3.5 VAE Modelinde Eğitim Süresinin (Epoch) Tahmin Tutarlılığı ve Sıra- lama Performansı Üzerindeki Etkisi	14
3.3.6 VAE İçin Katman Tasarımı	15

3.3.7	Z (Gizli Uzay) Katmanı	18
3.3.8	VAE Modelinin Eğitimi ve Performans Değerlendirmesi	19
3.3.9	VAE için NDCG@K Veri Hazırlama Aşaması	20
3.3.10	NDCG@K Veri Hazırlama Aşaması	21
3.3.11	VAE İçin Özelleştirilmiş Kayıp Fonksiyonu	22
3.4	Kullanıcı-Öğretilen Etkileşim Matrisinin Oluşturulması ve Veri Temsili	22
3.5	Kullanılan Veri Seti ve E-Ticaret Senaryosuna Uyarlanması	24
4	ARAŞTIRMA BULGULARI VE TARTIŞMA	26
5	SONUÇLAR	28
5.1	Karşılaşılan Zorluklar	30
5.2	Öneriler ve Gelecek Çalışmalar	31
	KAYNAKLAR	32

ŞEKİLLER DİZİNİ

Şekil 3.1	VAE Modelinin Katman Yapısı ve İşlem Akışı	17
Şekil 3.2	VAE Modelinde Loss-Epoch Grafiği	20

TABLolar DİZİNİ

Tablo 3.1	MF Faktör Sayısı Analiz Sonuçları (Regülasyon Katsayısı Hepsinde Sabit ve 0.05)	10
Tablo 3.2	MF Regularizasyon Analiz Sonuçları (Faktör Sayısı Hepsinde Sabit ve 300)	11
Tablo 3.3	VAE Modeli Öğrenme Oranı Analiz Sonuçları (Beta = 0.07, Epoch = 200, Dropout = 0,4 ,Latent=50, Int=200)	13
Tablo 3.4	VAE Modeli Mimari Kapasite Analiz Sonuçları (Beta = 0.07, LR = 0.0005, Epoch = 200, Dropout Oranı = 0.4)	13
Tablo 3.5	VAE Modeli Beta Parametresi Analiz Sonuçları (LR = 0.0005, Epoch = 200, Dropout Oranı = 0.4, Latent=50, Int=200)	13
Tablo 3.6	VAE Modeli Dropout Oranı Analiz Sonuçları (Beta = 0.01, LR=0.0005, Latent=50, Int=200)	14
Tablo 3.7	VAE Modeli Epoch Sayısına Bağlı Performans Değişimi (Beta = 0.01, Latent=50, Int=200, LR=0.0005, Dropout=0.4)	15
Tablo 3.8	Veri Seti Filtreleme Sonrası Genel İstatistikler	23
Tablo 3.9	MF ve VAE Modellerinin Örnek Tahmin Karşılaştırması	25
Tablo 4.1	VAE ve MF Modellerinin Performans Metrikleri Karşılaştırması	26
Tablo 4.2	MovieLens-20M Veri Seti Üzerinde Model Performans Karşılaştırmaları	27
Tablo 5.1	Deneylerin Gerçekleştirildiği Donanım Özellikleri	28
Tablo 5.2	VAE ve MF Modellerinin Performans Metrikleri Karşılaştırması	28

1 GİRİŞ

Öneri sistemleri, e-ticaret gibi platformlarda kullanıcı deneyimini iyileştirmek ve ticari başarıyı arttırmak açısından önemli bir yere sahiptir.

Bu çalışma, açık derecelendirme tahmini kapsamında, derin bir öğrenme yöntemi olan varyasyonel otokodlayıcı ile geleneksel bir yöntem olan matris faktörizasyonu modellerinin performanslarını deneysel olarak karşılaştırmayı amaçlamaktadır. Yapılan analizler, seyrek yapıya sahip bir kullanıcı-öge etkileşim matrisi üzerinde gerçekleştirilmiştir. Elde edilen sonuçlarla tahmin doğruluklarını ölçmek için kök ortalama kare hata, ortalama mutlak hata ve öneri listelerinin uygunluğunu ölçen normalize edilmiş indirgenmiş kümülatif kazanç metrikleri kullanarak incelenmiştir. Yapılan karşılaştırmalı değerlendirme sonuçlarına bakılarak iki modelleme yaklaşımının da derecelendirme tahmini bağlamındaki güçlü ve sınırlı yönlerinin olduğunu ortaya koymaktadır.

1.1 Literatür Taraması

Liang vd. [1] yaptığı bu çalışmada MF ile VAE modellerinin NDCG ve diğer metrikler üzerinden (Recall) sonuçlarını gösteriyor. Çalışmaya göre VAE modelinin bir türevi olan Mult-VAE modeli farklı veri setleri üzerinden (ML-20M, Netflix, MSD) diğer modellere (MF modelinin bir türevi olan WMF gibi) kıyasla daha iyi Recall (geri çağırma) ve NDCG sonucu vermiştir. Bu sonuçlar, VAE'nin sıralama başarısı açısından doğrusal modellerden daha üstün olduğunu kanıtlamaktadır.

Rajesh [2] tarafından yazılan makalede, MF yöntemi için kullanılan SVD ve diğer çeşitli MF modellerin RMSE, MAE ve diğer metrikler üzerinden kıyaslaması mevcuttur.

Li ve She [3] tarafından yapılan CVAE çalışmasında, VAE'nin türevi olan CVAE modelini diğer derin öğrenme ve geleneksel modeller ile farklı değerlendirme metrikleri üzerinden karşılaştırılmıştır.

Liang ve çalışma arkadaşlarının [4] Variational Autoencoders for Collaborative Filtering çalışmasında VAE mimarisine "çok terimli" (multinomial) bir olasılıksal hesap ekleyerek sistemi işbirlikçi filtreleme işi için daha da yetenekli hale getirdi. Çalışmada VAE'nin türevi olan Mult-

VAE ile diğer varyasyonel otokodlayıcılar ve matris faktörizasyonun türevi olan ağırlıklı matris faktörizasyonu (WMF) gibi geleneksel yöntemlerle farklı veri setleri ve metrikler üzerinden karşılaştırmalı tablo şeklinde sunmaktadır. Ayrıca diğer otokodlayıcı modellerini de kendi içlerinde karşılaştırmaktadır .

Tavsiye sistemleri alanındaki bir diğer çalışmalardan "RecoTour" serisinin üçüncü bölümünde Variational Autoencoders for Collaborative Filtering with Mxnet and Pytorch isimli çalışmada VAE mimarisi derinlemesine incelenmiştir. VAE modelinin kodlayıcı (encoder) ve kod çözücü (decoder) teknik uygulamalarına yer verilmiştir. Farklı veri setleri üzerinde yapılan deneylerde, öğrenme oranı (learning rate), optimum epoch sayısı gibi parametrelerin kayıp (loss) değerleri üzerindeki etkileri analiz edilmiştir. Ayrıca yapılan çalışmada farklı otokodlayıcı modellerinin performansları Recall, NDCG gibi metrikleri üzerinden karşılaştırmalı olarak tablo halinde sunulmuştur [5].

Fraihat vd. [6] tarafından yapılan bir diğer çalışmada da çok kriterli öneri sistemleri için VAE tabanlı bir yöntemi öneriyor. VAE modelinde katman tasarımı ve yöntemlerinden bahsedilmiştir. Yapılan çalışmada Yahoo ve MovieLens gibi veri setleri üzerinden MAE, RMSE metrikleriyle test edilmiştir. Elde edilen bulgular sonucunda bu yöntemin çok kriterli öneri sistemlerinde etkili olduğu saptanmıştır.

Bobadilla ve ekibi [7] tarafından yazılan diğer bir çalışmada öneri sistemlerinde kolaboratif filtreleme yöntemlerini geliştirmek için varyasyonel teknikleri derin modellere ekleniyor. Geleneksel derin öğrenme tabanlı kolaboratif filtreleme modelleri genellikle gizli uzayın süreksiz olması ve yapısal olmaması nedenlerinden yetersiz kalabilmektedir. Buna karşın VAE gibi modeller ise olasılıksal yaklaşımla gizli uzayları sağlam ve sürekli hale getirirler.

Liang vd. [8] tarafından yazılan derleme makalesinde öneri sistemleri alanlarında VAE tabanlı yöntemlerin kapsamlı bir şekilde literatür taraması yapılmıştır. Bu belgede özellikle VAE tabanlı yöntemleri sistematik olarak sınıflandırır ve özetler. VAE tabanlı modellerin geleneksel yöntemlere göre veri seyrekliği ve karmaşıklığı zorluklara karşı neden öne çıktığını açıklar. VAE'nin güçlü yönlerini, model mimarisini, avantajlarını ve kullanım alanlarını inceler. Bu belgede VAE'nin geleneksel kolaboratif filtreleme ve diğer derin öğrenme yöntemlerine göre

avantajları vurgulanır.

Tafsi [9] tarafından sunulan Medium yazısı İşbirlikçi Filtreleme (Collaborative Filtering - CF) ile MF yöntemini anlatan bir içeriktir. Makalede kullanıcı-öge matrisinde gizli uzayın çıkarılması gibi işbirlikçi filtreleme ile MF'nin temel mantığı açıklanır. MF yönteminin güçlü ve zayıf yönlerinden bahsedilir.

Alabduljabbar [10] tarafından yapılan çalışmada Riyad'daki restoranlar için öneri sistemi gerçekleştirilmiştir. Bu çalışmada MF tabanlı SVD gibi modeller geliştirmiştir. Bu yöntemler, işbirlikçi filtreleme kapsamında kullanıcı-restoran ilişkilerini modellemek için tercih edilmiştir. Modelin performansı RMSE, MAE gibi metriklerle değerlendirilmiştir. Elde edilen sonuçlarla SVD model özellikle RMSE açısıyla en iyi sonucu vermiştir.

2 KULLANILAN YAZILIMLAR ve YÖNTEMLER

Bu çalışma, 3.10 sürümüyle Python programlama dili ile geliştirilmiştir. Visual Studio Code (VS Code) geliştirme ortamıyla kurgulanmıştır.

2.1 Kullanılan Kütüphaneler

Bu çalışmada NumPy ve Pandas kütüphaneleri veri işleme ve sayısal hesaplamalar için kullanılmıştır. Pandas kütüphanesi ile veri setinin bulunduğu .dat dosyası okunur ve kullanıcı-öge bazında filtreleme yapar. ID'lerin sayısal indislere dönüştürmek ve sonuçları tablo halinde sunmak gibi işlemleri de gerçekleştirir. NumPy ise temel matris işlemleri, maskeleme, rastgele örnekleme, RMSE, MAE gibi hata metriklerini manuel hesaplama ve NDCG gibi metriklerde gerekli olan matematiksel işlemler için temel altyapı görevlerini üstlenir.

Veri seti büyük ve seyrek yapıda olduğu için SciPy kütüphanesi kullanılmıştır. Oluşturulan kullanıcı-öge puan matrisinde çoğu hücre boşluktan oluştuğu için, bu matrisin yoğun biçimde tutulması verimsiz olacaktır. Sıkıştırılmış Seyrek Satır (Compressed Sparse Row - CSR) formatı sayesinde yalnızca sıfır olmayan değerler saklanarak bellek kullanımı azaltılmıştır ve hesaplama verimlilik artmıştır.

Klasik öneri sistemi yaklaşımı olan MF modeli için de Surprise kütüphanesi kullanılmıştır. Bu kütüphane öneri sistemleri için geliştirilmiş Python kütüphanesidir. Yapılan çalışmada Reader sınıfında puan ölçeği tanımlanır, Dataset sınıfı ile veri hazırlanır ve SVD sınıfı kullanılarak MF modeli eğitilir.

Derin öğrenme tabanlı öneri sistemi modeli olan VAE için Tensorflow ve Keras kütüphaneleri kullanılmıştır. Bu kütüphanelerle VAE modelinin temel hesaplama altyapısı sağlanmıştır ve kod çözücü mimarisi kurulmuştur. Özel katman mantığı kullanılarak da örnekleme ve KL kaybı modele eklenmiştir. Modelin daha hızlı ve kararlı öğrenmesi amacıyla da Adam optimizasyon algoritması tercih edilmiştir.

Son olarak da Keras'ın arka uç (keras.backend ve keras.ops) ve rastgele sayıların üretilmesi için kullanılan keras.random modülüyle de VAE'nin yeniden parametreleme süreci için önemli rol oynamıştır. Bunun sayesinde gizli uzaydan örnekleme yapılabilirken modelin uçtan uca eği-

tilebilir olması sağlanmıştır. Böylece VAE modelinde hem yeniden yapılandırma kaybı hem KL kaybı birlikte optimize edilerek kullanıcı-öge etkileşimlerini daha iyi bir altyapıyla öğrenmektedir.

2.2 Veri Ön İşleme

Bu çalışmada verilerin ön işlenmesi, kullanıcı-öge etkileşim matrisinin oluşturulması ve modelleme aşamalarında Python tabanlı açık kaynaklı kütüphaneler kullanılarak oluşturulmuştur. Kullanıcı-öge bazlı minimum etkileşimler ile veri seyrekliği azaltılmış ve modelleme için daha dengeli bir veri yapısı elde edilmiştir

2.3 Geleneksel Yöntem

Geleneksel yöntem olarak MF yöntemi seçilmiştir. Bu modelin seçilme nedenleri olarak literatürlerde sıkça vurgulanan ölçeklenebilirlik (scalability), yorumlanabilirlik (interpretability), sparsity (seyrek veri) ile baş edebilme gibi güçlü yönleridir [9]. MF modelinde SVD algoritması uygulanmıştır. MF modeli, kullanıcı-öge temsillerini öğrenmek üzere 300 faktör gizli (latent) boyut ile eğitilmiştir. Aşırı öğrenmeyi sınırlandırmak için de düzenleme terimi 0.05 kullanılmıştır. MF modeli verinin %80'i ile eğitilmiş, %20'si ile de test edilmiştir.

2.4 Derin Öğrenme Yöntemi

Derin öğrenme yöntemi olarak VAE yöntemi kullanılmıştır. VAE yönteminin seçilme nedenleri olarak literatürlerde vurgulanan veri seyrekliği ve kompleks yapılarla baş etme, başka modellerle kolay birleştirebilme, derin öğrenme teknikleriyle kolayca entegre edilebilme gibi güçlü yönleridir [8]. Bu mimaride kodlayıcı kullanıcı-öge etkileşim matrisini girdi olarak alır ve bu bilgiyi sıkıştırır. Kodlayıcı aşamasının sonunda, gizli uzay dağılımını temsil eden ortalama vektörü ve logaritmik varyans değerleri üretilir. Bu değerler giriş olarak alınarak özel olarak oluşturulmuş "SamplingAndKL" katmanında z gizli uzay vektörü oluşturulur. Kod çözücü yapısı ise girdi olarak z gizli uzay vektörünü alır ve bu sıkıştırılmış bilgiyi yeniden inşa eder. Modelde gizli uzay boyutu 50, ara katman boyutu ise 200 olarak belirlenmiştir. VAE modeli verinin %80'i ile eğitilmiş, %20'si ile de test edilmiştir. VAE modelinin eğitimi sırasında maskeleme işlemi yapılarak yalnızca kullanıcı tarafından oy verilmiş ögeler üzerinden hata hesapla-

ması sağlayabilen "masked_MSE" özel kayıp fonksiyonu üretilmiştir. Optimizasyon sürecinde ise "Adam" algoritması tercih edilmiştir. Model eğitimi için epoch sayısı 200 belirlenmiştir.

2.5 Model Performans Metrikleri ve Hesaplamalar

RMSE ve MAE hem Surprise kütüphanesiyle hazır fonksiyonlarla hem de manuel olarak hesaplama ile modellerin tahmin performanslarını değerlendirmiştir. Surprise kütüphanesi dokümantasyonunda belirtilen standart formüller doğrultusunda hesaplanmıştır [11]. RMSE Denklem 2.1 ve MAE Denklem 2.2 metriklerinin formülleri aşağıda verilmiştir .

$$RMSE = \sqrt{\frac{1}{|\hat{R}|} \sum_{\hat{r}_{ui} \in \hat{R}} (r_{ui} - \hat{r}_{ui})^2} \quad (2.1)$$

RMSE, tahmin edilen değerler ile gerçek değerlerin farkının kareleri alınıp test veri kümesi içindeki toplam sayıya bölünür. Daha sonra da elde edilen ortalamanın karekökü alınma işlemidir.

$$MAE = \frac{1}{|\hat{R}|} \sum_{\hat{r}_{ui} \in \hat{R}} |r_{ui} - \hat{r}_{ui}| \quad (2.2)$$

MAE, gerçek değer ile tahmin edilen değerlerin mutlak farkının toplam veri setindeki örnek sayısına bölünmesi işlemidir.

Bu metriklere ek olarak da sıralama kalitesini ölçen özel bir NDCG@5 fonksiyonu tanımlanmıştır. NDCG metriği modelin sıralama başarısını ölçmektedir. Bu hesaplamada gerçek dünyada kullanıcılar ilk ürünlere baktığı gerekçeyle öneri listesi uzunluğu 5 olarak belirlenmiştir, gerçek derecelendirme değerleri için eşik değeri 4.0 olarak belirlenmiştir. 4.0 ve üzeri olan öğeler alakalı kabul edilmiştir. Eşik değerinin 4.0 seçilmesinin nedeni literatürlerde (Filipovic vd. yaptığı çalışmada [12]) de 4.0 eşik değeri seçilmiştir. NDCG metriğinin hesaplanması, literatürde kabul görmüş standart yöntemler ve güncel teknik dökümantasyonlar esas alınarak gerçekleştirilmiştir [13]. IDCG Denklem 2.4 ve NDCG Denklem 2.5 formülleri aşağıda verilmiştir.

- **İndirgenmiş Kümülatif Kazanç (DCG):**

$$DCG_k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} \quad (2.3)$$

- **İdeal İndirgenmiş Kümülatif Kazanç (IDCG):**

$$IDCG_k = \sum_{i=1}^{|REL|} \frac{rel_i^{ideal}}{\log_2(i+1)} \quad (2.4)$$

- **Normalleştirilmiş İndirgenmiş Kümülatif Kazanç (NDCG):**

$$NDCG_k = \frac{DCG_k}{IDCG_k} \quad (2.5)$$

İndirgenmiş Kümülatif Kazanç (DCG) değeri, alaka düzeylerinin sıralamadaki pozisyonlarına göre logaritmik olarak ağırlıklandırılmasıyla ($\log_2(i+1)$) hesaplanırken, aynı matematiksel prosedür ideal liste için de uygulanarak IDCG değeri elde edilmektedir. Son aşamada, DCG değerinin IDCG değerine oranlanmasıyla elde edilen normalize edilmiş skorlar bir listede toplanmakta ve tüm kullanıcıların ortalaması alınarak sistemin nihai sıralama başarısı raporlanmaktadır.

Çalışmada özel olarak oluşturulan masked_MSE Maskelenmiş Ortalama Kare Hata (Mean Squared Error - MSE) ise kullanıcının oy vermediği hücreleri eğitim dışında tutmak için yazılan bir fonksiyondur. MSE, modelin eğitiminde kullanılacak kayıp değerleri üretilmektedir. Sedhain vd. [14] tarafından önerilen yaklaşım temel alınmıştır. Kullanıcı başına hata kareleri toplamı Denklem 2.6, yığın ortalaması Denklem 2.7 formülleri aşağıda verilmiştir.

- **Kullanıcı Başına Hata Kareleri Toplamı (L_u):**

$$L_u = \sum_{i \in \mathcal{O}_u} (r_{ui} - \hat{r}_{ui})^2 \quad (2.6)$$

Denklem 2.6'daki formülde, kullanıcı bazlı bir hata skoru elde edilmektedir. Her bir kullanıcı için sadece o kullanıcının etkileşimde bulunduğu öğeler dikkate alınır. Daha sonra gerçek değerler ile modelin tahmin ettiği değerler arasındaki farkın karesi toplanmaktadır.

- **Yığın Ortalaması**

$$\mathcal{L}_{REC} = \frac{1}{N} \sum_{u=1}^N L_u \quad (2.7)$$

Denklem 2.7’deki formülde ise, yığın (batch) ortalaması hesaplanmaktadır. Veri setindeki ya da ilgili yığındaki bütün kullanıcıların bireysel hata skorlarının toplamı kullanıcı sayısına bölünür.

VAE modelinin matematiksel altyapısı, Kingma ve Welling [15] tarafından önerilen Auto-Encoding Variational Bayes çalışmasına dayanmaktadır.

Çalışmada gizli uzayda gizli değişkenler z Denklem 2.8 ile üretilir:

$$z = \mu + \sigma \cdot \epsilon \quad (2.8)$$

Burada epsilon standart normal dağılımdan gelir ve μ ve σ ile ölçeklenip kaydırılır.

Çalışmada VAE modeli için özel oluşturulan SamplingAndKL katmanında geçen KL kaybı formülü Denklem 2.9’da verilmiştir.

$$\mathcal{L}_{KL}^{\beta} = \beta \cdot \left(-\frac{1}{2} \sum_{i=1}^d (1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2) \right) \quad (2.9)$$

$\mathcal{L}_{KL}^{\beta}$, modelin öğrendiği gizli değişken dağılımının standart normal dağılımdan (0 ortalama ve 1 varyans) ne kadar saptığını ölçerek sistemi düzenlileştirir (regularization); formüldeki μ_i^2 terimi gizli uzayın ortalama değerini, σ_i^2 varyans değerini, d ise gizli uzayın boyut sayısını temsil eder.

3 VAE ve MF Tabanlı Karşılaştırmalı Öneri Sistemi

Bu çalışmanın temel amacı, VAE gibi derin öğrenme tabanlı yaklaşımların öneri sistemlerindeki performansını; MF gibi geleneksel modelleme yöntemleriyle akademik ölçütler üzerinden karşılaştırmalı olarak değerlendirmektir.

3.1 Veri Ön İşleme ve Matris Oluşturma

Algoritma 1: Seyrek Verinin Azaltılması Amacıyla Kullanıcı ve Ürün Filtreleme İşlemi

```
1 min_user_ratings = 10
2 user_counts = df['user_id'].value_counts()
3 valid_users = user_counts[user_counts >= min_user_ratings].index
4 df = df[df['user_id'].isin(valid_users)].copy()
5
6 min_item_ratings = 5
7 item_counts = df['item_id'].value_counts()
8 valid_items = item_counts[item_counts >= min_item_ratings].index
9 df = df[df['item_id'].isin(valid_items)].copy()
```

Bu kod bloğuna göre, bir kullanıcının veri setinde kalabilmesi için en az 10 öğeye oy vermiş olması, bir öğenin veri setinde kalabilmesi için en az 5 kullanıcıdan oy almış olması gerekir. Bu işlem için her kullanıcının kaç değerlendirme yaptığı ve her öğenin kaç kullanıcı tarafından değerlendirme yapıldığı value_counts() fonksiyonu ile sayılır. Eşik değerler altında kalan değerler, kullanıcı-öge veri setinden silinir. Eşik değerler 1. ve 6. satırlarda belirtilmiştir. Böylece kullanıcı ve öge bazında aşırı seyrek olan etkileşimler temizlenir ve veri setindeki seyreklik azalmış olur.

Algoritma 2: Kullanıcı-Ürün İndeksleme ve Etkileşim Matrisi Oluşturma

```
1 user_to_index = {user: i for i, user in enumerate(df['user_id'].unique())}
2
3 df['user_index'] = df['user_id'].map(user_to_index).astype(np.int32)
4 item_to_index = {item: i for i, item in enumerate(df['item_id'].unique())}
5
6 df['item_index'] = df['item_id'].map(item_to_index).astype(np.int32)
7
8 num_users = df['user_index'].nunique()
9 num_items = df['item_index'].nunique()
10 full_sparse_matrix = csr_matrix((df['rating'], (df['user_index'], df['item_index'])), shape=(num_users, num_items))
```

Bu kod bloğunda, filtrelenmiş kullanıcı kimlikleri (user id) 0,1,2, ... şeklinde dizine ekler ve yeni bir sütun ekleyip atanan indis numaralarına göre yazar. Aynı işlemler öğeler için de gerçekleşir. Bu işlem enumerate() ve sözlük yapısı (user_to_index ve item_to_index) kullanılarak yapılır; user_id ve item_id sütunları map() fonksiyonuyla bu indislere güncellenir. Daha sonra da filtrelenmiş kullanıcıların toplam sayısı ve filtrelenmiş öğelerin toplam sayısı nunique() fonksiyonu ile kaydedilir. CSR matris formatıyla da kullanıcılar satır öğeler sütun olacak şekilde filtrelenmiş kullanıcı-öge matrisi oluşturulur. Bu yapılandırma ile de veri setinde olan kullanıcı ve ürün kimlikleri modelin anlayabileceği tam sayı indekslerine dönüştürülmesi sağlanır. Bu yapılan dönüşüm sayesinde kullanıcı-öge etkileşim matrisi, işbirlikçi filtreleme algoritmalarının temelini oluşturur. Bunlar sayesinde MF ve VAE modellerinin performans değerleri için uygun bir zemin hazırlanır.

3.2 Matris Faktörizasyonu Modeli

MF modeli eğitilirken regülasyon katsayısı değeri 0.05 tutularak değiştirilen faktör sayıları ve performans değerlendirme metrikleri sonuçları Tablo 3.1’de verilmiştir.

Tablo 3.1: MF Faktör Sayısı Analiz Sonuçları (Regülasyon Katsayısı Hepsinde Sabit ve 0.05)

Faktör Sayısı	RMSE	MAE	NDCG@5
50	0.8728	0.6903	0.8713
100	0.8685	0.6867	0.8738
150	0.8669	0.6853	0.8724
200	0.8660	0.6848	0.8743
250	0.8662	0.6846	0.8747
300	0.8646	0.6836	0.8741
350	0.8650	0.6838	0.8752

Deneysel sonuçlara bakılarak faktör sayısı sabitken regülasyon katsayısı değerleri değiştirilerek en iyi performans için en düşük RMSE ve MAE değeri için 300 faktör sayısı seçilmiştir.

MF modeli eğitilirken faktör sayısı 300 tutularak değiştirilen regülasyon katsayısı değeri ve performans değerlendirme metrikleri sonuçları Tablo 3.2’de verilmiştir.

Faktör sayısı sabitken en iyi RMSE, MAE ve NDCG değeri regülasyon katsayısı değeri 0.05 iken belirlenmiştir.

Tablo 3.2: MF Regularizasyon Analiz Sonuçları (Faktör Sayısı Hepsinde Sabit ve 300)

Reg Değeri (reg_all)	RMSE	MAE	NDCG@5
0.01	0.9081	0.7125	0.8497
0.05	0.8646	0.6836	0.8741
0.1	0.8878	0.7049	0.8628
0.2	0.9135	0.7284	0.8489

3.2.1 MF Modeli Eğitimi ve Performans Metrikleri

Algoritma 3: Matris faktörizasyonu Eğitim

```

1 trainset, testset = train_test_split(data, test_size=0.2, random_state
    =42)
2 svd_model = SVD(n_factors=300, reg_all=0.05, random_state=42, verbose=
    False)
3 svd_model.fit(trainset)
4 predictions_mf = svd_model.test(testset)
5 rmse_mf = rmse(predictions_mf, verbose=False)
6 mae_mf = mae(predictions_mf, verbose=False)
7 ndcg_mf = get_ndcg_at_k(predictions_mf, k=5, threshold=4.0)

```

Bu kod bloğunda regülasyon katsayısı (reg_all) parametresi ile aşırı öğrenme engellenir. Bu parametre, modelin ağırlıklarının büyüklükleriyle orantılı bir şekilde ceza terimi uygulayarak modelin karmaşıklığını kontrol altına alır ve modelin veriler üzerinde tahmin yeteneğini artırır. Kod bloğuna göre veri seti train_test_split() fonksiyonu ile %20 test verisi %80 eğitim verisi olarak veri seti ayrılmıştır. random_state=42 ile bölme işleminin tekrarlanabilirliği sağlanır. SVD sınıfı kullanılarak da model oluşturulur. Model için 300 gizli faktör (n_factors=300) sayısı, kullanıcı ve öğeler için regülasyon katsayısı 0.05 (reg_all=0.05) ve rastgelelik kontrolü (random_state=42) belirlenmiştir. Model fit() fonksiyonu ile eğitim (train) veri seti üzerinde eğitilir ve test veri kümesi üzerinden test() fonksiyonu ile değerlendirme tahminleri yapılır (predictions_mf). Tahminlerin doğruluğu rmse() ve mae() fonksiyonlarıyla yapılır. RMSE ve MAE hesaplamaları, surprise kütüphanesinden alınan rmse ve mae fonksiyonlarıyla hesaplanmıştır. Ayrıca NDCG hesabı özel oluşturulan get_ndcg_at_k() fonksiyonuyla hesaplanmıştır. Bu fonksiyonda öneri listesi boyutu 5, eşik değer ise 4.0 verilmiştir.

3.2.2 MF için NDCG@K Veri Hazırlama Aşaması

Algoritma 4: Sıralama Metrikleri İçin Tahmin Verilerinin Kullanıcı Bazlı Gruplandırılması

```
1 def get_ndcg_at_k(predictions_or_matrix, k=5, threshold=4.0, is_vae=False
2 ):
3     if not is_vae:
4         top_n = defaultdict(list)
5         for uid, iid, true_r, est, _ in predictions_or_matrix:
6             top_n[uid].append((iid, true_r, est))
7         dataset = top_n
```

Bu kod bloğu, `is_vae` parametresi `False` olduğu durumda çalışır. Yapılan çalışmada MF modelinden elde edilen tahmin çıktıların NDCG hesaplamasına uygun bir veri yapısına dönüştürür. Daha sonra kullanıcı kimliği altında demetin 0. elemanı ürün kimliği, demetin 1. elemanı gerçek değer (true rating), demetin 2. elemanı modelin tahmin ettiği değer (estimated) şeklinde demet oluşturulmuştur. Modelin performans değerlendirilmesinde `k` değişkeni ilk 5 öneriye odaklanmasını sağlar. Çalışmada 5 verilmiştir. Oylamada kullanıcının öğeyi beğenmesi için verilen puan eşik değeri 4.0 belirlenmiştir.

3.3 Varyasyonel Otokodlayıcı Modeli

VAE modelinin performansını test etmek amacıyla; öğrenme oranı, katman boyutları, beta parametresi ve dropout oranı üzerinde deneysel çalışmalar gerçekleştirilmiştir.

3.3.1 Öğrenme Oranı (Learning Rate) Analizi

Deneysel sonuçlarda (beta değeri = 0.07, epoch değeri = 200, dropout değeri = 0.4, gizli katman boyutu = 50, ara katman boyutu = 200) en düşük RMSE, MAE değeri ve en yüksek NDCG değeri ile en ideal denge öğrenme oranı 0.0005 değerinde sağlanmıştır. Tablo 3.3'te öğrenme oranı değişikliğine göre performans metriklerinin değerleri verilmiştir.

Tablo 3.3: VAE Modeli Öğrenme Oranı Analiz Sonuçları (Beta = 0.07, Epoch = 200, Dropout = 0.4 ,Latent=50, Int=200)

Öğrenme Oranı	RMSE	MAE	NDCG@5
0.0001	0.9670	0.7746	0.8891
0.001	0.9592	0.7677	0.8903
0.0005	0.9570	0.7644	0.8928

3.3.2 Mimari Kapasite ve Gizli Boyut Analizi

Deneyler sonucunda (beta değeri = 0.07, epoch değeri = 200, öğrenme oranı = 0.0005, dropout değeri = 0.4) en düşük RMSE ve MAE değeri ile en ideal sonuç ara katman 200, gizli boyut 50’de sağlanmıştır. Ayrıca gizli boyutun 100 olarak seçilmesi NDCG@5 skorunun yükselmesinde etkili olmuştur. Tablo 3.4’te katman boyutları değişikliğine göre performans metriklerinin değerleri verilmiştir.

Tablo 3.4: VAE Modeli Mimari Kapasite Analiz Sonuçları (Beta = 0.07, LR = 0.0005, Epoch = 200, Dropout Oranı = 0.4)

Ara Katman	Gizli Boyut	RMSE	MAE	NDCG@5
200 (Referans)	50	0.9570	0.7644	0.8928
512	50	0.9918	0.7889	0.8738
200	100	0.9695	0.7723	0.8993
512	100	1.0063	0.8008	0.8833

3.3.3 Beta Parametresi ve Düzenleştirme Etkisi

Yapılan deneyler sonucunda (öğrenme oranı = 0.0005, epoch sayısı =200, dropout oranı = 0.4, gizli uzay katmanı boyutu = 50, ara katman boyutu = 200) KL kaybının ağırlığını kontrol eden beta parametresi, en düşük RMSE, MAE ve en yüksek NDCG@5 skoru ile en dengeli sonuç beta değeri 0.01 iken sağlanmıştır. Beta parametresi değiştirilerek yapılan analizler tablosu Tablo 3.5’te gösterilmiştir.

Tablo 3.5: VAE Modeli Beta Parametresi Analiz Sonuçları (LR = 0.0005, Epoch = 200, Dropout Oranı = 0.4, Latent=50, Int=200)

Beta (β)	RMSE	MAE	NDCG@5
0.01	0.9492	0.7598	0.8987
0.07	0.9570	0.7644	0.8928
0.1	0.9627	0.7703	0.8972
0.2	0.9553	0.7649	0.8887

3.3.4 Dropout Oranı ile Model Genelleştirme

Dropout değişkeni, eğitim sürecinde nöronların belirli bir kısmını rastgele ve geçici süre devre dışı kalmasını sağlar . Dropout analizi Tablo 3.6’da gösterilmiştir. Yapılan deneyler sonucunda (beta değeri = 0.01, öğrenme oranı = 0.0005, gizli uzay boyutu = 50, ara katman boyutu = 200) en düşük RMSE, MAE ve en yüksek NDCG@5 skoru ile en dengeli sonuç dropout değeri 0.4 ile sağlanmıştır.

Tablo 3.6: VAE Modeli Dropout Oranı Analiz Sonuçları (Beta = 0.01, LR=0.0005, Latent=50, Int=200)

Dropout Oranı	RMSE	MAE	NDCG@5
0.2	0.9940	0.7945	0.8902
0.4	0.9492	0.7598	0.8987
0.5	0.9627	0.7736	0.8989

3.3.5 VAE Modelinde Eğitim Süresinin (Epoch) Tahmin Tutarlılığı ve Sıralama Performansı Üzerindeki Etkisi

Tablo 3.7’de yer alan veriler incelendiğinde, en düşük RMSE, MAE değerleri ile en dengeli sonuç epoch sayısı 200 iken belirlenmiştir. Ayrıca 100 epoch sayısında NDCG@5 skorunun en iyi değeri hesaplanmıştır.

Tablo 3.7: VAE Modeli Epoch Sayısına Bağlı Performans Değişimi (Beta = 0.01, Latent=50, Int=200, LR=0.0005, Dropout=0.4)

Epoch	RMSE ↓	MAE ↓	NDCG@5 ↑
50	1,0059	0,8221	0,8988
100	0,9512	0,7701	0,9034
150	0,9535	0,7647	0,8999
200	0,9492	0,7598	0,8987
250	0,9570	0,7640	0,8946
300	0,9776	0,7822	0,8892

3.3.6 VAE İçin Katman Tasarımı

Algoritma 5: VAE İçin Gizli Uzay ve Katman Tasarımı

```

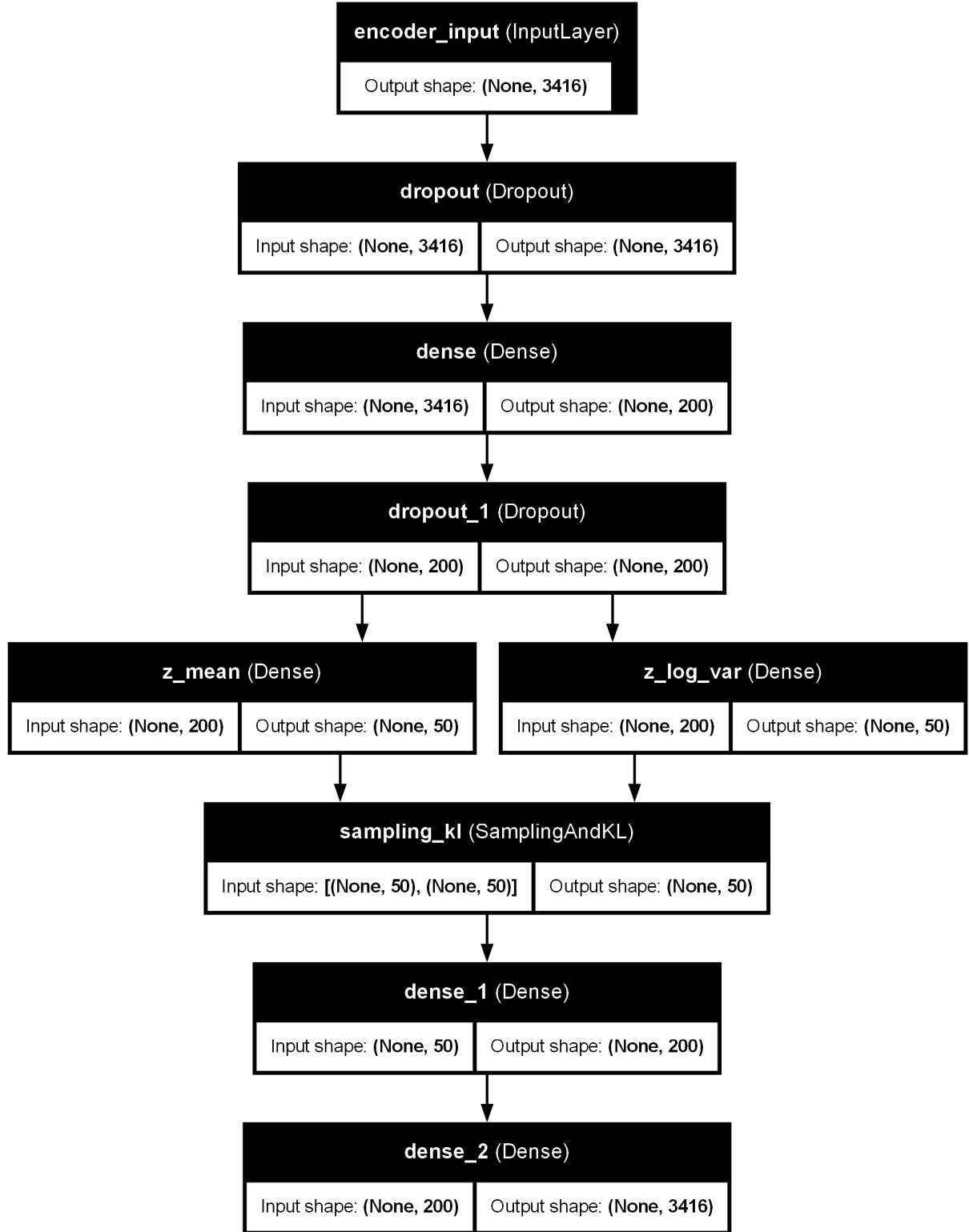
1 original_dim = num_items
2 latent_dim = 50
3 intermediate_dim = 200
4 dropout_rate = 0.4
5
6 input_layer = Input(shape=(original_dim, ), name='encoder_input')
7 h = Dropout(dropout_rate)(input_layer)
8 h = Dense(intermediate_dim, activation='relu')(h)
9 h = Dropout(dropout_rate)(h)
10 z_mean = Dense(latent_dim, name='z_mean')(h)
11 z_log_var = Dense(latent_dim, name='z_log_var')(h)
12
13 z = SamplingAndKL(beta=0.01, name='sampling_kl')([z_mean, z_log_var])
14
15 decoder_h = Dense(intermediate_dim, activation='relu')
16 decoder_output = Dense(original_dim, activation=None)
17
18 h_decoded = decoder_h(z)
19 x_decoded_mean = decoder_output(h_decoded)

```

Bu kod bloğunda, girdi katmanı (input_layer) filtrelenmiş öğe sayısına eşit (original_dim = num_items) olacak şekilde tanımlanmıştır. Kodlayıcı kısmından önce aşırı öğrenmeyi (overfitting) engellemek amacıyla dropout yöntemi kullanılmıştır, ardından ara katmanda veriler relu aktivasyon fonksiyonu ile işlenir. Daha sonra tekrardan dropout uygulanır. Gizli uzay boyutu

(latent_dim) 50 olarak tanımlanmıştır. Ara katman boyutu (intermedite_dim) 200 nöron olarak tanımlanmıştır. Kodlayıcı katmanı çıktı olarak z_log_var (varyansın logaritması) ve z_mean (ortalama) üretir. SamplingAndKL katmanı ile bu iki parametreyi kullanarak gizli uzaydan rastgele bir z örneği çeker. Bu verinin sıkıştırılmış halidir. Kod çözücü katmanı da sıkıştırılmış bu z bilgisi girdi olarak alır ve bu sıkıştırılmış bilginin çözümüleme işlemini yapar.

Modelinin katman yapısı ve işlem akışı Şekil 3.1’de gösterilmiştir.



Şekil 3.1: VAE Modelinin Katman Yapısı ve İşlem Akışı

3.3.7 Z (Gizli Uzay) Katmanı

Algoritma 6: VAE Modeli İçin Özel Kayıp Bileşeni ve Örnekleme Katmanı

```
1
2 class SamplingAndKL(Layer):
3     def __init__(self, beta=0.01, **kwargs):
4         super().__init__(**kwargs)
5         self.beta = beta
6
7     def call(self, inputs):
8         z_mean, z_log_var = inputs
9
10        batch = ops.shape(z_mean)[0]
11        dim = ops.shape(z_mean)[1]
12
13        epsilon = random.normal(shape=(batch, dim))
14
15        kl_loss = -0.5 * ops.mean(1 + z_log_var - ops.square(z_mean) -
16                                ops.exp(z_log_var), axis=-1)
17
18        self.add_loss(self.beta * ops.mean(kl_loss))
19        return z_mean + ops.exp(0.5 * z_log_var) * epsilon
```

Bu kod bloğunda Keras kütüphanesinden miras alan SamplingAndKL adında bir katman üretilmiştir. call fonksiyonu çalıştırıldığında kodlayıcı bölümünden elde edilen z_log_var ve z_mean kullanılır. Önce küçük parçalar boyutu (batch) ve gizli boyut (dim) belirlenir. Sonra bu boyutlarda standart normal dağılımdan (epsilon) rastgele örnekler üretilir. KL kaybı normal dağılımdan ne kadar saptığı 15. satırdaki formül ile hesaplanmıştır. Hesaplamadan sonra sonuç ile beta değeri çarpılır ve modelin toplam kaybına eklenerek z örneği çekilir ($z = z_mean + \exp(0.5 * z_log_var) * epsilon$). Bu formül ile de ortalama ve standart sapma ile gizli temsil üretilir.

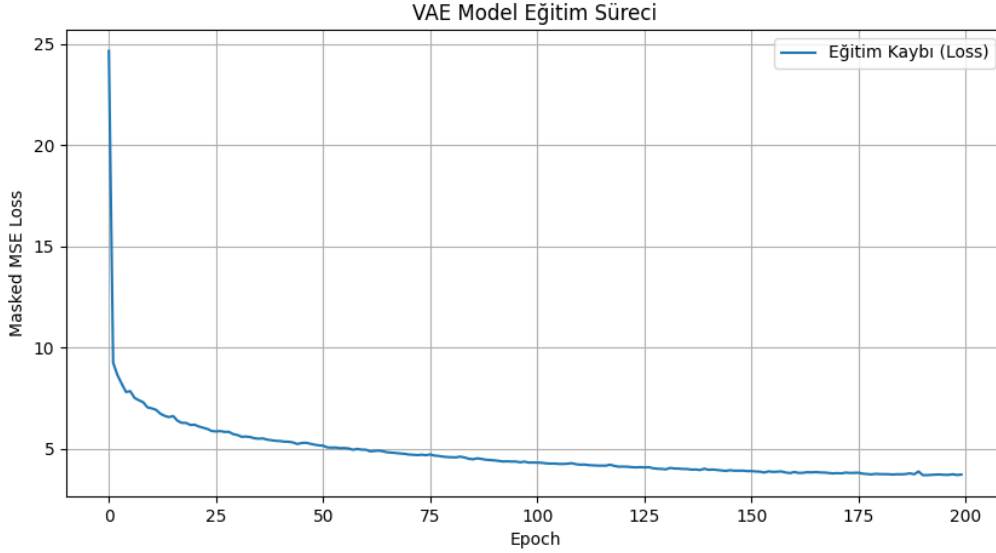
3.3.8 VAE Modelinin Eğitimi ve Performans Değerlendirmesi

Algoritma 7: VAE Modelinin Eğitimi ve Tahmin Performansının Değerlendirilmesi

```
1 vae = Model(input_layer, x_decoded_mean)
2 optimizer = Adam(learning_rate=0.0005)
3 vae.compile(optimizer='adam', loss=masked_mse)
4 new_epochs = 200
5 vae.fit(R_train_normalized, R_train_normalized, epochs=new_epochs,
        batch_size=128, shuffle=True, verbose=1)
6 R_predicted_vae = vae.predict(R_test_dense / 5.0)
7 observed_mask = (R_test_dense > 0)
8 rmse_vae = np.sqrt(np.sum(((R_test_dense - R_predicted_vae * 5.0) *
        observed_mask)**2) / np.sum(observed_mask))
9 mae_vae = np.sum(np.abs(diff)) / np.sum(observed_mask)
10 ndcg_vae = get_ndcg_at_k((R_test_dense, R_predicted_vae), k=5, threshold
        =4.0, is_vae=True)
```

Bu kod bloğunda, giriş katmanı ile kod çözücü çıkışı bir araya getiren VAE model fonksiyonu tanıtılmaktadır. Kodda ilk önce `vae.compile()` fonksiyonu ile VAE modeli derlenir. Öğrenme oranı 0.0005 olarak belirlenmiştir. Kayıp fonksiyonu olarak VAE modeli için özel oluşturulmuş `masked_mse` fonksiyonu kullanılmaktadır. Epoch sayısı 200 girilerek modelin öğrenme süreci başlatılmaktadır. Modelin üreteceği tahmin puanları, test verisindeki kullanıcı etkileşimlerini temel alan bir tahminleme ve hata analizi yapma süreciyle gerçekleştirilmektedir. Modelin ürettiği tahmin puanları, test verisindeki mevcut oyların konumlarını belirleyen bir maskeleme matrisi aracılığıyla filtrelenerek sadece gerçek gözlemlerle kıyaslanmaktadır. Sonrasında, model `fit()` fonksiyonu ile eğitim verisi için `R_train_normalized`, 128 batch boyutu ve veriler karıştırılarak 200 epoch boyunca eğitilir. Eğitim tamamlandıktan sonra, `predict()` ile test verisi olan `R_test_dense` ve normalize edilmiş çıktılarımız `R_predicted_vae` matrisinde saklanır. Daha sonra, test verisinde gözlemlenen değerlerin bir maskesi olan `observed_mask` oluşturulur; bu maske yalnızca gerçek kullanıcılar ve öğeler arasındaki etkileşimleri seçmek için kullanılır. Bu kapsamda hesaplanan RMSE, tahminlerin gerçek değerlerden karesel farkını ölçerek büyük hatalara karşı hassas bir performans göstergesi sunarken; MAE ise sistemin genel mutlak hata payını temsil etmektedir. Hata metriklerine ek olarak kullanılan NDCG@5 metriği, modelin en yüksek puan alacağı tahmin edilen ilk 5 ürünü ne kadar doğru sıraladığını bir eşik değeri (th-

reshold=4.0) üzerinden analiz etmektedir. VAE modelinin eğitiminden sonra loss-epoch grafiği Şekil 3.2’de gösterilmiştir.



Şekil 3.2: VAE Modelinde Loss-Epoch Grafiği

Şekil 3.2’de sunulan loss-epoch grafiği incelendiğinde, VAE modelinin eğitim kaybının ilk epoklarda hızla azaldığı ve 150. epok civarında kararlı bir seviyeye ulaştığı görülmektedir. Bu durum, modelin MovieLens veri seti üzerindeki gizli karakteristikleri başarıyla öğrendiğini ve aşırı öğrenmeye kaçmadan optimum noktaya yaklaştığını göstermektedir.

3.3.9 VAE için NDCG@K Veri Hazırlama Aşaması

Algoritma 8: VAE Çıktılarının Sıralama Metrikleri İçin Kullanıcı Bazlı Yapılandırılması

```

1 def get_ndcg_at_k(predictions_or_matrix, k=5, threshold=4.0, is_vae=
    False)
2 else:
3     R_test_dense, R_predicted_vae = predictions_or_matrix
4     num_users_test, num_items = R_test_dense.shape
5     dataset = defaultdict(list)
6
7     for user_index in range(num_users_test):
8         observed_items = np.where(R_test_dense[user_index, :] > 0)[0]
9
10        for item_index in observed_items:

```

```

11         true_r = R_test_dense[user_index, item_index]
12         est = R_predicted_vae[user_index, item_index]
13         dataset[user_index].append((item_index, true_r, est))

```

Bu kod bloğu, `is_vae` parametresi `true` olduğu durumda çalışır. Giriş olarak `R_test_dense` (test veri kümesi) ve `R_predicted_vae` (VAE modelinin derecelendirme tahminleri) matris çiftini alır. İlk başta test kümesindeki kullanıcı ve öge sayısı matrisin boyutlarından alınır. Daha sonra veri kümesi içerisinde olan her kullanıcı için `dataset` adında sözlük oluşturulur. 5.satırda gösterilmiştir. Her kullanıcının (`user_index`) test veri setinde değerlendirme yaptığı öğeler seçilir. 8. satırda gösterilmiştir. Her model için gerçek derecelendirme puanı (`true_r`) ve tahmin edilen puan (`est`) alınır. Bu edinilen veriler (`item_index, true_r, est`) kullanıcıların kimliği altında listeye eklenir. Sonuç olarak `dataset` sözlüğü, her kullanıcı için NDCG hesaplamasında kullanılacak item, gerçek puan ve tahmin değeri üçlüsünden oluşan bir veri yapısı sağlar.

3.3.10 NDCG@K Veri Hazırlama Aşaması

Algoritma 9: Tavsiye Başarısının NDCG@K Metriği ile Değerlendirilmesi

```

1     ndcgs = []
2     for uid, user_ratings in dataset.items():
3         user_ratings.sort(key=lambda x: x[2], reverse=True)
4         relevance = [1.0 if r[1] >= threshold else 0.0 for r in
                    user_ratings[:k]]
5         ideal_relevance = sorted([1.0 if r[1] >= threshold else 0.0 for r
                    in user_ratings], reverse=True)[:k]
6         dcg = sum(rel / math.log2(i + 2) for i, rel in enumerate(
                    relevance)) #DCG=toplam i=0 dan k-1 e kadar reli/log2(i+2)
7         idcg = sum(rel / math.log2(i + 2) for i, rel in enumerate(
                    ideal_relevance))
8         if idcg > 0:
9             ndcgs.append(dcg / idcg)
10    return np.mean(ndcgs) if ndcgs else 0.0

```

Kod, veri seti içerisindeki her kullanıcı (`uid`) için çalışır. Öncelikle, kullanıcının tüm değerlendirmeleri (`user_ratings`), modelin tahmin ettiği değerlere göre (`est`) azalan sırada sıralanır. Ardından, gerçek puan (`true rating`) eşik değeri (`threshold`) ile karşılaştırılarak alaka düzeyi (re-

levance) oluşturulur; eşik değeri üzerindeyse 1, altında ise 0 atanır. Aynı işlem tüm öğeler için ideal sıralama (ideal_relevance) olarak yapılır ve ilk k değeri alınır. Kullanıcının sıralanmış öneri listesinin DCG değeri toplanarak hesaplanır. Benzer şekilde, ideal sıralamanın DCG'si hesaplanır. Eğer IDCG sıfırdan büyükse, kullanıcının NDCG değeri $DCG/IDCG$ olarak belirlenir ve tüm kullanıcılar için listeye eklenir. Fonksiyon, kullanıcılar üzerindeki NDCG değerlerinin ortalamasını alır; eğer kullanıcılar yoksa 0 döndürür.

3.3.11 VAE İçin Özelleştirilmiş Kayıp Fonksiyonu

Algoritma 10: Maskelenmiş Ortalama Kare Hata Fonksiyonu

```
1 def masked_mse(y_true, y_pred):
2     mask = ops.cast(ops.not_equal(y_true, 0), dtype=K.floatx())
3     squared_error = ops.square(y_true - y_pred) * mask
4     reconstruction_loss_per_user = ops.sum(squared_error, axis=-1)
5     return ops.mean(reconstruction_loss_per_user)
```

Bu kod bloğu, VAE modelinin eğitiminde kullanılan MSE fonksiyonunu tanımlamaktadır. Burda MSE tanımlanırken oy verilmiş öğeler üzerinden hesaplama yapılmaktadır. Kodun 2. satırında gösterilmiştir. Daha sonra maskelenmiş veriler matematiksel hesaplama için uygun bir maske matrisine dönüştürülmektedir. Ardından, gerçek değerler ile tahmin edilen değerler arasındaki farkın karesi alınıp maske ile çarpılmaktadır. Kodun 3. satırında gösterilmiştir. Bu işlemin yapılmasının amacı oy verilmemiş öğelerin hata payı sıfırlanır ve sadece gerçek etkileşimi olan veriler üzerinden bir hata matrisi elde edilmektedir. Bu hesaplama sürecinin devamında da her bir kullanıcı için toplam hata satır bazında toplanır ve bu yöntem ile yeniden yapılandırma kaybı ile KL kaybı arasındaki ölçek dengesinin korunmasını sağlamaktadır. Kodun 4. satırında gösterilmiştir. Son olarak, tüm kullanıcıların hata ortalamaları alınır. Modelin eğitiminde kullanılacak kayıp değerleri üretilmektedir. Kodun 5. satırında gösterilmiştir.

3.4 Kullanıcı-Öğesi Etkileşim Matrisinin Oluşturulması ve Veri Temsili

Algoritma 11: Filtreleme Sonrası Kullanıcı-Öğesi Matris Boyutlarının Hesaplanması

```
1 num_users_final = R_dense.shape[0]
2 num_items_final = R_dense.shape[1]
3
```



```

4 print(f"Filtreleme Sonrası Kullanıcı Sayısı (Satır): {num_users_final}")
5 print(f"Filtreleme Sonrası Ürün/Öge Sayısı (Sütun): {num_items_final}")

```

Bu kod bloğunda filtreleme işleminden sonra oluşan yoğun matriste kullanıcı-öge sayısı belirlenir ve ekrana yazdırılır. `num_users_final = R_dense.shape[0]` satırı matristeki satırların sayısını alarak kullanıcı sayısını, `num_items_final = R_dense.shape[1]` satırında ise matristeki sütun sayısını alarak öge sayısını verir ve `print()` fonksiyonuyla ekrana yazdırma işlemini gerçekleştirir.

Filtreleme adımlarının ardından elde edilen veri, satırlarda kullanıcıların ve sütunlarda ögelerin yer aldığı bir kullanıcı-öge matrisi biçiminde temsil edilmiştir. Bu matris yapısında her bir hücre, ilgili kullanıcının söz konusu öge için verdiği derecelendirme puanını ifade etmektedir. Kullanıcının etkileşimde bulunmadığı ögeler için ise hücre değeri sıfır olarak bırakılmıştır.

Oluşturulan bu matris, veri seyrekliğini ve bellek kullanımını optimize etmek amacıyla başlangıçta seyrek matris formatında tutulmuş, VAE modelinin gereksinimleri doğrultusunda daha sonra yoğun (dense) forma dönüştürülerek model eğitiminde kullanılmıştır. Tablo 3.8’de filtreleme sonucu kalan kullanıcı ve ürün sayıları gösterilmiştir.

Tablo 3.8: Veri Seti Filtreleme Sonrası Genel İstatistikler

Özellik	Sayı / Değer
Kullanıcı Sayısı (Satır)	5624
Ürün/Öge Sayısı (Sütun)	3413

3.5 Kullanılan Veri Seti ve E-Ticaret Senaryosuna Uyarlanması

Bu çalışmada, MovieLens veri seti kullanılmıştır. MovieLens, kullanıcıların belirli öğelere verdikleri açık sayısal derecelendirmeleri içeren, öneri sistemleri literatüründe yaygın olarak tercih edilen bir veri setidir. Veri setinin sütunları sırasıyla kullanıcı kimliği, öğe kimliği, verilen derecelendirme puanı ve etkileşimin gerçekleştiği zamana ait zaman damgası bilgisinden oluşmaktadır.

Çalışmada kullanılan veri seti, "kullanıcıID::ögeID::puan::zaman" formatında yapılandırılmıştır. Kullanıcı-öge etkileşimlerini satır bazında temsil etmektedir. Gerçek derecelendirme puanları 1 ile 5 arasında değişir. Zaman damgası bilgisi ise modelleme sürecine dahil edilmiştir. Veri setinde olan kullanıcı kimliği, öğe kimliği, gerçek derecelendirme puanı, MF modelinin tahmin ettiği derecelendirme puanı, VAE modelinin tahmin ettiği derecelendirme puanı ilk 10 satır için Tablo 3.9’da verilmiştir. Bu tablo incelendiğinde, MF modeli yöntemi özellikle düşük puanlı etkileşimlerde (örneğin Kullanıcı 4812) gerçek değere yakın değerler üretmiştir. VAE modeli ise bazı örneklerde yüksek sayısal sapmalar gösterse de yüksek puanlı öğeleri tespit etme ve sıralama eğiliminde (örneğin Kullanıcı 3687) sergilediği performans, derin öğrenme modelinin olasılıksal yapısının sunduğu NDCG başarısını destekler niteliktedir. VAE modelinin gizli uzay temsilleri sayesinde kullanıcı tercihlerini tahmin etmede geleneksel MF yaklaşımıyla rekabet edilebilecek düzeyde ve hatta NDCG açısından daha yetkin bir profile sahip olduğu saptanmıştır.

Tablo 3.9: MF ve VAE Modellerinin Örnek Tahmin Karşılaştırması

Kullanıcı ID	Film ID	Gerçek Puan	MF Tahmini	VAE Tahmini
5472	1367	2,0	2,77	3,19
398	1805	2,0	3,41	3,46
670	2069	3,0	3,21	3,63
3687	1580	5,0	2,84	3,76
3312	3702	1,0	3,08	3,32
4812	1126	1,0	2,40	2,06
1842	1321	4,0	4,16	3,80
3716	1552	4,0	3,85	2,84
2004	1527	4,0	4,25	3,48
3335	1197	4,0	4,43	4,09

MovieLens veri seti film öneri senaryosu için tasarlanmıştır. İçerdiği kullanıcı-öge etkileşim yapısı bakımından e-ticaret platformlarında karşılaşılan kullanıcı-ürün derecelendirme problemleriyle büyük ölçüde örtüşmektedir. Bu nedenle, bu veri seti yapılan çalışmada temsili bir e-ticaret senaryosu olarak ele alınmıştır. Veri setindeki filmler ürün, kullanıcılar ise müşteri olarak yorumlanmıştır. Bu yaklaşım, öneri algoritmalarının davranışlarını gerçek e-ticaret ortamlarına genellenebilir biçimde değerlendirmeye olanak sağlamaktadır.

4 ARAŞTIRMA BULGULARI VE TARTIŞMA

Tablo 4.1: VAE ve MF Modellerinin Performans Metrikleri Karşılaştırması

Model	RMSE ↓	MAE ↓	NDCG@5 ↑
VAE	0,9207	0,7569	0,9091
MF	0,8646	0,6836	0,8741

Bu çalışmada, geleneksel bir yöntem olan MF modeli ile derin öğrenme yöntemi olan VAE modellerinin derecelendirme tahmini performansları üretilen kullanıcı–öğe etkileşim matrisi üzerinde incelenmiştir. Veri seti olarak MovieLens kullanılmıştır. Veriler seyrek yapıya dönüştürülmüştür. Düşük etkileşimli kullanıcılar ve öğeler modeller için kullanılmamıştır.

Deneysel sonuçlar Tablo 4.1’de gösterilmiştir. MF modeli düşük RMSE ve MAE ile daha isabetli tahminler üretirken, VAE modeli ise yüksek NDCG@5 değeri ile yüksek sıralama kalitesi sergilemiştir.

MF yaklaşımları, SVD tabanlı yapıları sayesinde, kullanıcı ve öğe ilişkilerini düşük boyutta gizli temsiller şeklinde modelleme yaparak derecelendirme tahmininde düşük hatalı sonuçlar üretebilmektedir. Bu özellik ile de MF modelinin RMSE ve MAE metrikleri üzerinden VAE modeline göre daha başarılı bir performans sergilemesini açıklayan etkenlerden biridir.

VAE tabanlı model ise daha yüksek RMSE ve MAE değerlerine sahiptir. Bunun sebebi olasılıksal gizli uzay yapısının yanı sıra da KL teriminin yeniden yapılandırma süreci üzerinde oluşturduğu düzenleyici etki ile ilişkilendirilebilir. VAE modellerinin bu özellikleri kullanıcı tercihlerini doğrusal olmayan ilişkiler üzerinden temsil edebilme yeteneği, özellikle sıralama odaklı değerlendirme ölçütlerinde avantaj sağlayabilmektedir. Elde edilen NDCG@5 sonuçları, VAE modelinin sıralama performansında MF modelini geride bırakarak daha başarılı bir tavsiye kalitesi sunduğunu göstermektedir. VAE modelinin ulaştığı 0.9091 değeri, puan tahmin hataları (RMSE/MAE) daha düşük olan MF modeline (0.8741) kıyasla, kullanıcılar için kritik olan ilk 5 önerinin sıralamasında VAE’nin daha efektif bir öğrenme gerçekleştirdiğini kanıtlamaktadır. Bulgularımız, Liang vd. [16] tarafından ortaya konulan VAE sıralama tabanlı metriklerde (NDCG) geleneksel matris faktörizasyonu yöntemlerinden daha üstün performans sergilediği teziyle de tutarlılık göstermektedir. İlgili literatürde sonuçlar Tablo 4.2’de gösterilmiştir.

Tablo 4.2: MovieLens-20M Veri Seti Üzerinde Model Performans Karşılaştırmaları

Model	Recall@20	Recall@50	NDCG@100
Mult-VAE (PR)	0.395	0.537	0.426
Mult-DAE	0.387	0.524	0.419
WMF (Weighted MF)	0.360	0.498	0.386
SLIM	0.370	0.495	0.401
CDAE	0.391	0.523	0.418

NDCG değerlerinin yapılan çalışmada literatürdeki değerlerden yüksek olmasının nedenleri; literatürdeki çalışmada, modelin başarısı tüm katalog içerisindeki binlerce öge arasından doğru olanı bulma (ranking) kapasitesine göre ölçülürken, yapılan çalışmada sadece kullanıcının zaten etkileşime girdiği öğeleri kendi içinde ne kadar iyi sıraladığına odaklanmaktadır. Bu durum, yapılan çalışmada ilgili kodun (`observed_items = np.where(R_test_dense[user_index, :] > 0)[0]`) satırıyla açıklanabilmektedir. Buna göre hesaplama kümesi sadece kullanıcının oy verdiği öğelerle sınırlandırılır. Yani hesaplamanın tüm katalog yerine yalnızca test setindeki gözlemlenen etkileşimler üzerinden yapılması nedeniyle skorların literatürdeki 'tüm öğeleri sıralama' (all-item ranking) testlerine göre daha yüksek çıktığı görülmektedir.

5 SONUÇLAR

Geliştirilen VAE ve MF modellerinin eğitim süreçlerinde kullanılan sistem bileşenleri Tablo 5.1’de sunulmuştur.

Tablo 5.1: Deneylerin Gerçekleştirildiği Donanım Özellikleri

Donanım Bileşeni	Özellikler
İşlemci	Intel Core i5-11400H
Bellek	16 GB DDR4
Ekran Kartı	NVIDIA GeForce RTX 3050 Ti
Depolama (SSD)	476 GB

Tablo 5.1’de belirtilen donanım mimarisi üzerinde gerçekleştirilen eğitim süreçlerinde; MF ve VAE modellerinin toplam eğitim süresi yaklaşık 3 dakika sürmüştür. Eğitim işlemleri ekran kartı (GPU) desteğine ihtiyaç duyulmadan, işlemci (CPU) üzerinde verimli bir şekilde tamamlanmıştır.

Bu çalışmada geleneksel bir yöntem olan MF ve derin öğrenme yöntemlerinden biri olan VAE modelleri kullanılmıştır. Veri seti olarak MovieLens kullanılmış olup bir kullanıcı-öğe etkileşim matrisi üzerinden karşılaştırmalı olarak sonuçlar değerlendirilmiştir. Uygulanan filtreleme işlemleri sonucunda veri seti 5624 kullanıcı ve 3413 öğeden oluşacak şekilde inşa edilmiştir.

Yapılan deneyler sonucunda (VAE modeli 3 kez eğitildikten sonra sonuçlar tabloya ortalaması alınarak girilmiştir.) MF modelinin RMSE ve MAE değerleri daha düşük çıkmıştır. Bu bulgular sonucunda geleneksel bir yöntem olan MF modelinin VAE modeline göre derecelendirme tahmin doğruluğunun daha başarılı sonuçlar verdiği saptanmıştır. Tablo 5.2’de gösterilmiştir.

Tablo 5.2: VAE ve MF Modellerinin Performans Metrikleri Karşılaştırması

Model	RMSE ↓	MAE ↓	NDCG@5 ↑
VAE	0,9207	0,7569	0,9091
MF	0,8646	0,6836	0,8741

Buna karşılık VAE modelinin MF modeline kıyasla NDCG@5 parametresi üzerinden daha başarılı sonuç verdiği görülmüştür. Bu sonuç, VAE modelinin doğrusal olmayan olasılıksal yapısının, en alakalı öğeleri listenin üst sıralarına yerleştirme kapasitesinde MF’ye karşı bir avantaj sağladığını kanıtlamaktadır.

Genel olarak deęerlendirildięinde ise MF modeli RMSE ve MAE metriklerinde VAE modeline gre daha başarılı sonuçlar verdięi iin VAE modeline gre tahmin doęruluęu aısından daha stn bir performans sunar. VAE modeli ise NDCG metrięinde MF modeline gre daha başarılı sonuçlar verdięi iin MF modeline kıyasla neri sıralamalarında daha başarılı sonuçlar retir.

5.1 Karşılaşılan Zorluklar

Bu çalışmanın yapılması sürecinde hem veri yapısından hem de kullanılan yöntemlerin gereksinimlerinden kaynaklanan çeşitli zorluklarla karşılaştırılmıştır. Özellikle kullanılan veri seti seyrek olduğundan modellerin öğrenme süreçlerinde zorluklarla karşılaşmıştır.

İlk olarak, MovieLens veri seti oldukça seyrek. Bu özellikle geleneksel ve karmaşık bir model olan VAE modelinin eğitimini zorlaştırmıştır. Bu nedenle, yeterli sayıda etkileşime sahip olmayan kullanıcı ve öğeler veri setinden elenmiştir. Bu işlemle veri seti boyutu küçülürken aynı zamanda modelin öğrenme süreci de daha kararlı hale gelmiştir. İkinci olarak özellikle VAE modelinde birçok parametre olmasından ötürü bu parametreleri denemek ve nihai denge sonucuna ulaşmak zor olmuştur. MF modelinde faktör sayısı ve regülasyon katsayısı değeri, VAE modelinde ise beta parametresi, dropout oranı, öğrenme oranı, epoch sayısı gibi parametrelerin fazla olmasından ötürü bunları deneyip denge performansını yakalamak zorlayıcı bir süreç olmuştur.

Son olarak da performans metrikleri değerlendirilmesi aşamasında RMSE, MAE ve NDCG gibi farklı özelliklere sahip metriklerin birlikte yorumlanması dikkat gerektirmiştir. RMSE ve MAE sayısal tahmin doğruluğunu kendine özgü yöntemlerle ölçerken NDCG sıralama kalitesini değerlendirmektedir. Bunun nedeni olarak modeller bir metrikte üstün, bir metrikte diğerine göre düşük sonuçlar verebilmiştir. Elde edilen bu sonular tek boyutlu değil, uygulamanın senaryosuna bağlı olarak ele alınmasını zorunlu kılmıştır.

5.2 Öneriler ve Gelecek Çalışmalar

Bu çalışma kapsamında elde edilen sonuçlar, farklı modelleme yaklaşımlarının öneri sistemleri üzerindeki etkilerini ortaya koymakla birlikte, ilerleyen çalışmalara yönelik çeşitli geliştirme alanları da sunmaktadır.

Öncelikle veri setini değiştirerek ya da arttırarak sonuçların daha iyi vermesi sağlanabilir. Seyrek veri yapısı ile yüksek veri yapısı karşılaştırması özellikle VAE gibi derin öğrenme tabanlı modelleri doğrudan etkileyecektir.

VAE modelinin performansı, mimari derinlik arttırılarak, gizli boyut sayısının yeniden ayarlanmasıyla ve beta parametresini farklı değerlerle tekrardan test edilmesi yoluyla iyileştirebilir. Özellikle de KL kaybı teriminin etkisi daha dengeli hale gelirse yeniden yapılandırma hatasının azalmasında katkı sağlayabilecektir.

Ayrıca, bu çalışmada sadece gerçek derecelendirme verileri kullanılmıştır. Gelecek çalışmalarda ek bilgiler (tür, zaman bilgisi, kullanıcı profili gibi) de modele eklenerek hibrit öneri sistemleri geliştirilebilecektir. Bu tür fazla verinin olması özellikle VAE tabanlı modellerde genelleme performansının arttırması beklenmektedir.

Son olarak da farklı değerlendirme metrikleriyle (Precision@K, Recall@K, MAP gibi) de modellerin performansının ölçülmesi ,elde edilen sonuçların daha da genellenebilirliğini güçlendirecektir. Bu yönelimle de derin öğrenme tabanlı yöntemler ile geleneksel tabanlı yöntemlerin birlikte ele alınacağı karşılaştırmalardaa öneri sistemleri altında önemli katkılar sağlayacağı düşünülebilir.

KAYNAKLAR

- [1] Liang, D., Krishnan, R. G., Hoffman, M. D., & Jebara, T. (2018). Variational Autoencoders for Collaborative Filtering. *Proceedings of the 2018 World Wide Web Conference (WWW '18)*, 689–698. Eriřim adresi: <https://arxiv.org/abs/1802.05814>.
- [2] Rajesh, D. B., & Kumar, A. (2025). Collaborative filtering models: An experimental and detailed comparative study. *Scientific Reports*, 15, 31667. Eriřim adresi <https://www.nature.com/articles/s41598-025-15096-4>
- [3] Li, X., & She, J. (2017). Collaborative variational autoencoder for recommender systems. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 305–314. <https://dl.acm.org/doi/epdf/10.1145/3097983.3098077>
- [4] Liang, D., Krishnan, R. G., Hoffman, M. D., ve Jebara, T. (2018). *Variational Autoencoders for Collaborative Filtering*. Proceedings of the 2018 World Wide Web Conference (WWW '18), 689–698. Eriřim adresi: <https://arxiv.org/abs/1802.05814> (Eriřim Tarihi: 4 Ocak 2026).
- [5] Chauhan, A. S. (2019). Recotour III: Variational autoencoders for collaborative filtering with MXNet and PyTorch. *Towards Data Science*. <https://medium.com/data-science/recotour-iii-variational-autoencoders-for-collaborative-filtering-with-mxnet-and-pytorch-710c924fa2fd>
- [6] S. Fraihat, Q. Shambour, M. A. Al-Betar ve S. N. Makhadmeh, “Variational Autoencoders-Based Algorithm for Multi-Criteria Recommendation Systems,” *Algorithms*, c. 17, s. 12, s. 561, 2024. <https://doi.org/10.3390/a17120561>
- [7] J. Bobadilla, F. Ortega, A. Gutiérrez ve Á. González-Prieto, “Deep variational models for collaborative filtering-based recommender systems,” *Neural Computing and Applications*, c. 35, ss. 7817–7831, 2023. <https://doi.org/10.1007/s00521-022-08088-2>
- [8] S. Liang, Z. Pan, W. Liu, J. Yin ve M. de Rijke, “A Survey on Variational Autoencoders in Recommender Systems,” *ACM Computing Surveys*, 2024. Eriřim Adresi: <https://staff>.

fnwi.uva.nl/m.derijke/wp-content/papercite-data/pdf/liang-2024-survey.pdf

- [9] O. Tafsi, “5 mins Recommender systems: Matrix Factorization Collaborative Filtering,” Medium, May 10, 2024. Eriřim Adresi: https://medium.com/@omar.tafsi_46401/5-mins-recommender-systems-matrix-factorization-collaborative-filtering-8eead5fbe18c
- [10] R. Alabduljabbar, “Matrix Factorization Collaborative-Based Recommender System for Riyadh Restaurants: Leveraging Machine Learning to Enhance Consumer Choice,” *Applied Sciences*, c. 13, s. 17, s. 9574, 2023. <https://doi.org/10.3390/app13179574>
- [11] Hug, N. (2020). Surprise: A Python library for recommender systems. *Journal of Open Source Software*, 5(52), 2174. <https://surprise.readthedocs.io/en/stable/accuracy.html>
- [12] M. Filipovic, B. Mitrevski, D. Antognini, E. Lejal Glaude, B. Faltings ve C. Musat, “Modeling Online Behavior in Recommender Systems: The Importance of Temporal Context,” *CEUR Workshop Proceedings*, Vol. 2955, 2021. Eriřim adresi: https://ceur-ws.org/Vol-2955/paper4.pdf?utm_source=chatgpt.com
- [13] Evidently AI. (2025). *Normalized Discounted Cumulative Gain (NDCG) Explained: Ranking and Recommendation Metrics Guide*. Eriřim adresi: <https://www.evidentlyai.com/ranking-metrics/ndcg-metric> (Eriřim Tarihi: 4 Ocak 2026).
- [14] Sedhain, S., Menon, A. K., Sanner, S., ve Xie, L. (2015). *AutoRec: Autoencoders Meet Collaborative Filtering*. WWW ’15 Companion: Proceedings of the 24th International Conference on World Wide Web, 111–112. Eriřim adresi: <https://doi.org/10.1145/2740908.2742726>.
- [15] Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*. <https://arxiv.org/pdf/1312.6114>
- [16] D. Liang, R. G. Krishnan, M. D. Hoffman, ve T. Jebara, “Variational autoencoders for collaborative filtering,” *Proceedings of the 2018 World Wide Web Conference (WWW ’18)*, ss. 689–698, 2018. <https://doi.org/10.1145/3178876.3186150>